算法分析

算法分析

- 算法分析的目的是要预测算法的行为，包括时间复杂性和空间复杂性，从而判断算法的优劣。

时间复杂性

- 决定程序运行时间的因素：
  1) 问题规模（输入规模）
  2) 代码质量
  3) 计算机系统性能
  4) 算法的时间复杂性

时间复杂性

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3 nsec	0.01 μ	0.02 μ	0.06 μ	0.51 μ	0.26 μ
16	4 nsec	0.02 μ	0.06 μ	0.26 μ	4.10 μ	65.5 μ
32	5 nsec	0.03 μ	0.16 μ	1.02 μ	32.7 μ	4.29 sec
64	6 nsec	0.06 μ	0.38 μ	4.10 μ	262 μ	5.85 cent
128	0.01 μ	0.13 μ	0.90 μ	16.38 μ	0.01 sec	10^{20} cent
256	0.01 μ	0.26 μ	2.05 μ	65.54 μ	0.02 sec	10^{58} cent
512	0.01 μ	0.51 μ	4.61 μ	262.14 μ	0.13 sec	10^{135} cent
2048	0.01 μ	2.05 μ	22.53 μ	0.01 sec	1.07 sec	10^{598} cent
4096	0.01 μ	4.10 μ	49.15 μ	0.02 sec	8.40 sec	10^{1214} cent
8192	0.01 μ	8.19 μ	106.50 μ	0.07 sec	1.15 min	10^{2447} cent
16384	0.01 μ	16.38 μ	229.38 μ	0.27 sec	1.22 hrs	10^{4913} cent
32768	0.02 μ	32.77 μ	491.52 μ	1.07 sec	9.77 hrs	10^{9845} cent
65536	0.02 μ	65.54 μ	1048.6 μ	0.07 min	3.3 days	10^{19709} cent
131072	0.02 μ	131.07 μ	2228.2 μ	0.29 min	26 days	10^{39438} cent
262144	0.02 μ	262.14 μ	4718.6 μ	1.15 min	7 mnths	10^{78894} cent
524288	0.02 μ	524.29 μ	9961.5 μ	4.58 min	4.6 years	10^{157808} cent
1048576	0.02 μ	1048.60 μ	20972 μ	18.3 min	37 years	10^{315634} cent

时间复杂性

Time complexity function	With present computer	With computer 100 times faster	With computer 1000 times faster
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31.6N_2$
n^3	N_3	$4.64N_3$	$10N_3$
n^5	N_4	$2.5N_4$	$3.98N_4$
2^n	N_5	$N_5 +6.64$	$N_5 +9.97$
3^n	N_6	$N_6 +4.19$	$N_6 +6.29$

渐近时间复杂性

- 评估算法性能并不需要对其执行时间作出准确统计，关心相对性能
- 在问题规模较小的情况下，不同算法求解同一问题的时间虽然会有差异，但可以忽略不计
- 我们最关心的是大规模问题的性能
- 我们更为关心的是在大规模的输入下时间复杂性增长趋势的差异，因此在比较两个算法时间复杂性时，通常是在输入规模n趋向无穷大的情形下进行讨论的，即进行渐近分析。
- 那么如何度量和评价算法执行时间的总体变化趋势和增长速度呢？

符号O

- 如果存在常数c和N，使得对于所有的n≥N，有g(n)≤cf(n)，则称函数g(n)相对于另一函数f(n)是O(f(n))的(读作“O”或者“大O”)。对于足够大的n，函数g(n)不超过函数f(n)的一个固定倍数。函数g(n)可能比cf(n)小，甚至是非常小；符号O仅仅是从上限来约束它。
- 如果g(n)=O(f(n))，通常f(n)的形式比g(n)简洁，但仍然能够反映g(n)的增长趋势

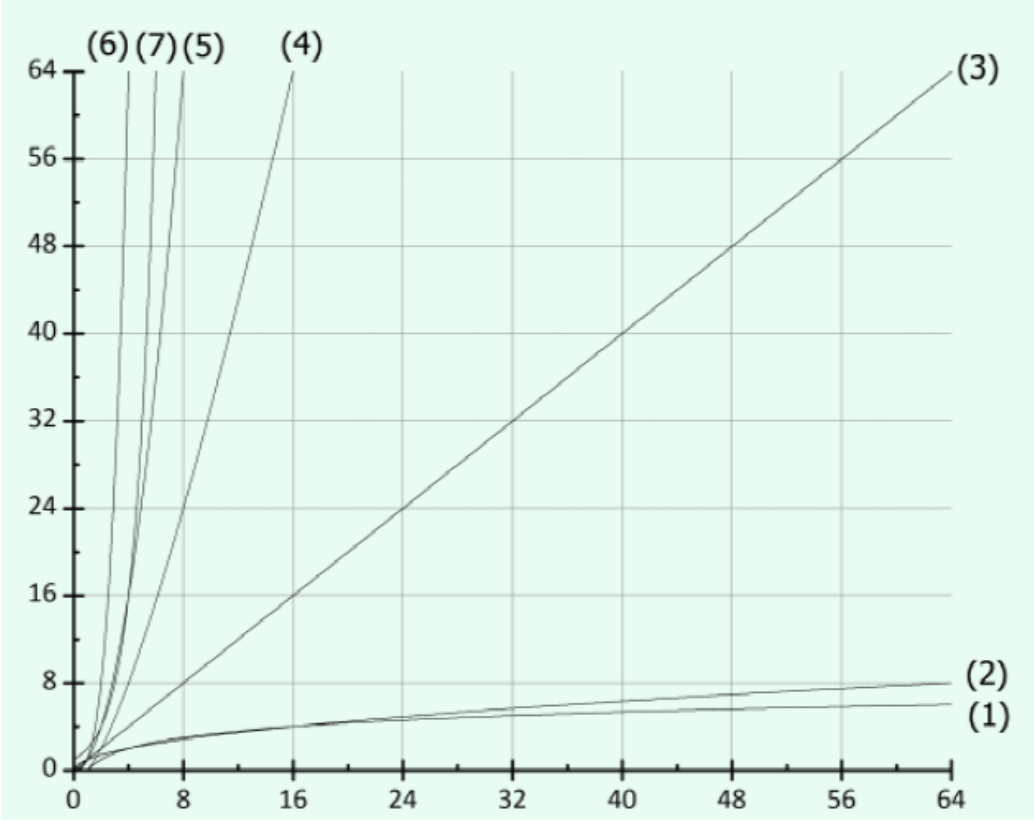
N的意义：
f1(n)=n^2
f2(n)=2n+20
f2=O(f1)
f1!=O(f2)
但是n<=5时f1(n)<f2(n)

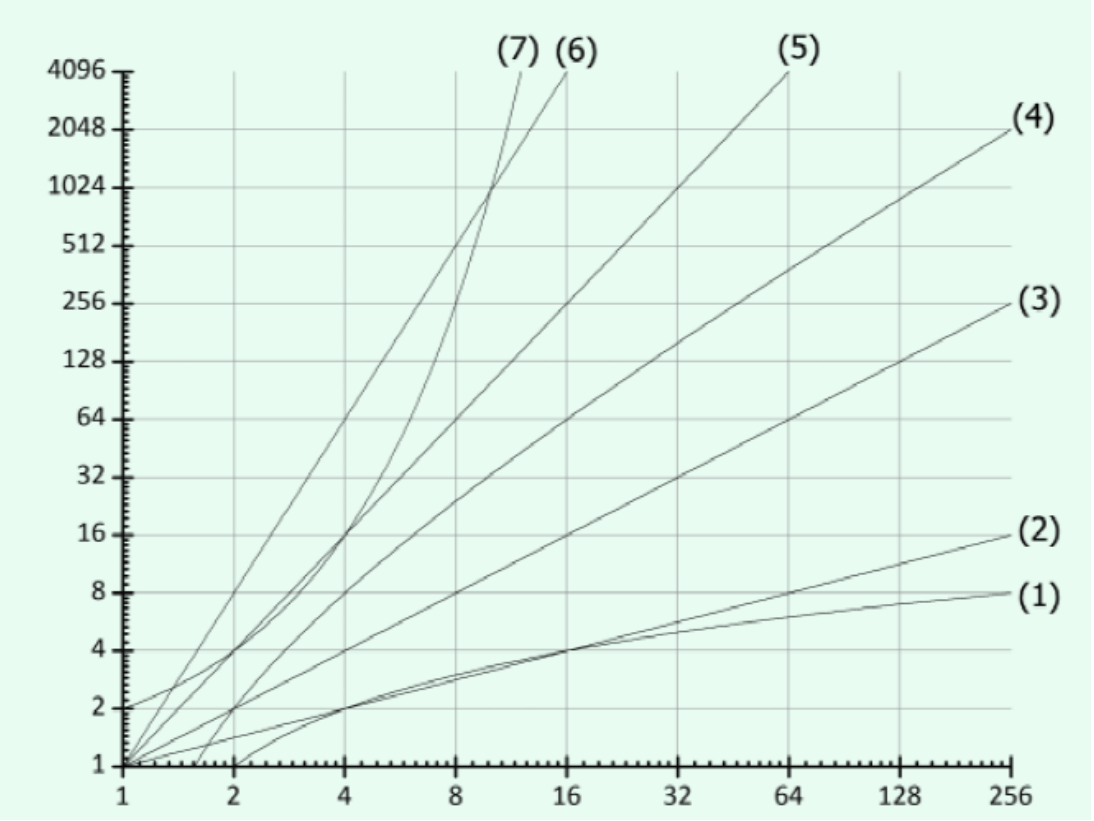
符号O

- 定理3.1：对于所有的常数c>0和a>1，以及所有的单调递增函数f(n)，有$(f(n))^c=O(a^{f(n)})$。即一个指数函数要比一个多项式函数增长得快。
- 引理3.2：1）如果f(n)=O(s(n))并且g(n)=O(r(n))，则f(n)+g(n) = O(s(n)+r(n))。2）如果f(n)=O(s(n))并且g(n)=O(r(n))，则f(n)g(n) = O(s(n)g(n))。
- 如果f(n)=O(g(n))，则O(f(n)+g(n))=O(g(n))
- 如果常数c>0，则O(c*f(n))=O(f(n))
- 如果常数a>b>0，则O(n^a+n^b)=O(n^a)
- 由于符号O对应于关系“≤”，不能对其定义减法和除法运算。也就是说，通常f(n) = O(s(n))和g(n) = O(r(n))并不意味f(n) - g(n) = O(s(n) - r(n))，或者f(n)/g(n) = O(s(n)/r(n))

不同的O(f(n))

1) O(logn)
2) O(sqrt(n))
3) O(n)
4) O(nlogn)
5) O(n^2)
6) O(n^3)
7) O(2^n)





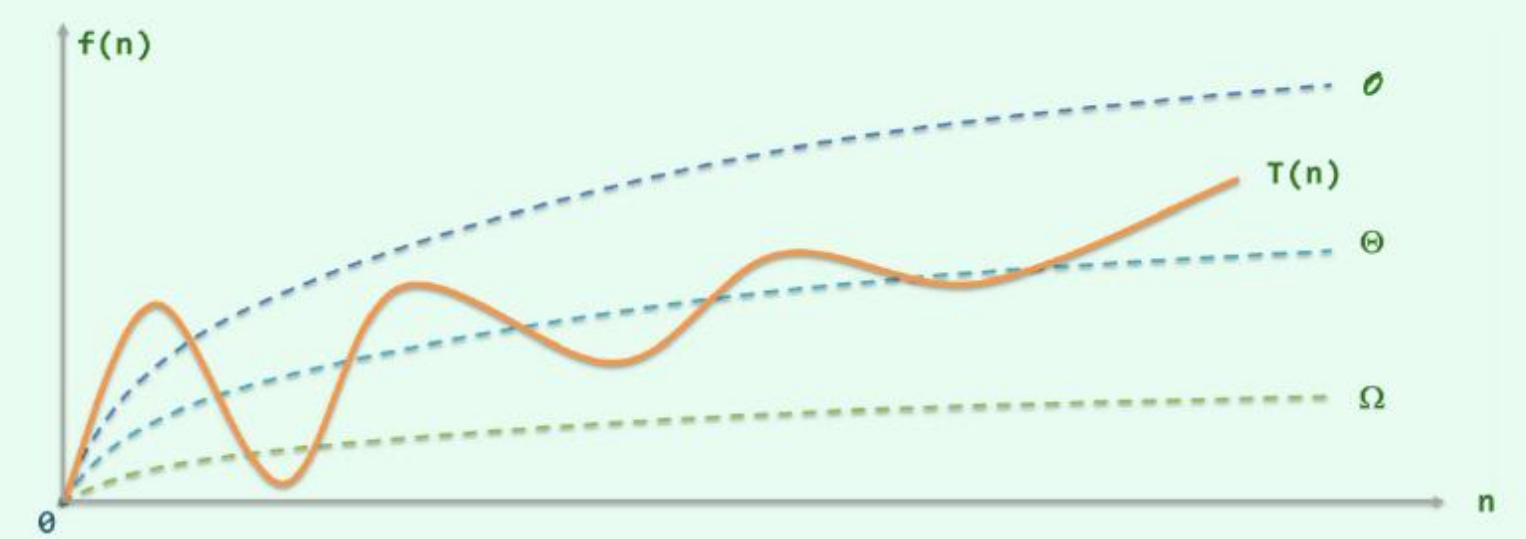
符号Ω

- 如果存在常数c和N，使得对于所有的$n \geq N$，有$g(n) \geq cf(n)$，则称函数g(n)相对于另一函数f(n)是$\Omega(f(n))$的。

- $f(n)=\Omega(g(n))$当且仅当$g(n)=O(f(n))$

符号Θ

- 如果存在常数 c_1，c_2 和 N，使得对于所有的 $n \geq N$，有 $c_1 f(n) \leq g(n) \leq c_2 f(n)$，则称函数g(n)相对于另一函数f(n)是Θ(f(n))的。
- $f(n)=\Theta(g(n))$当且仅当$f(n)=\Omega(g(n))$，$f(n)=O(g(n))$



符号o

- 如果对于任意的c>0，可以找到N>0，使得对于所有的n≥N，有g(n)<cf(n)，则称函数g(n)相对于另一函数f(n)是o(f(n))
- 如果$\lim_{n\to\infty} f(n)/g(n)=0$，则f(n)=o(g(n))
- f(n)=o(g(n))当且仅当f(n)=O(g(n))但g(n)≠O(f(n))
- 定理3.3：对于所有的常数c>0和a>1，以及所有的单调递增函数f(n)，有$(f(n))^c=o(a^{f(n)})$，即一个指数式函数要比一个多项式函数增长得快。

$$1 \prec \log\log n \prec \log n \prec \sqrt{n} \prec n^{3/4} \prec n \prec n\log n \prec n^2 \prec 2^n \prec n! \prec 2^{n^2}$$

空间复杂性

- 空间复杂度表明的是运行一个算法所需要的临时存储空间。
- 不把输入和输出需要的空间作为空间复杂度的一部分，也不对程序自身占用的空间计数
- $S(n)=O(T(n))$
- 空间可以复用，时间不能复用。例如用二进制表示法从0到2^n-1计数，时间为指数，空间为多项式
- 时间与空间的平衡
- 虽然$T(n)$是$S(n)$的一个上界，但是在一些对空间效率要求较高或者输入规模巨大的场合还是需要进行更加精确的空间复杂性分析

最优算法

- 通常如果我们能够证明某一问题的时间复杂性为$\Omega(f(n))$， 那么任一能够在$O(f(n))$时间内求解该问题的算法即为最优算法

如何估计算法运行时间

- 计算迭代次数
- 计算基本运算的频度
- 使用递推关系

基本运算

- 所谓基本运算是指具有最高频度的运算，其它运算的频度最多为基本运算频度的常数倍

Algorithm 1 INSERTIONSORT
Input: An array A[1..n] of n elements
Output: A[1..n] sorted in nondecreasing order

```
for i←2 to n
    x←A[i]
    j←i-1
    while (j>0) and (A[j]>x)
        A[j+1]←A[j]
        j←j-1
    end while16
    A[j+1]←x
endfor
```

Algorithm 2 MODINSERTIONSORT
Input: An array A[1..n] of n elements
Output: A[1..n] sorted in nondecreasing order

```
for i←2 to n
    x←A[i]
    k←MODBINARYSEARCH(A[1..i-1],x)
    for j ← i-1 downto k
        A[j+1]←A[j]
    endfor
    A[k]←x
endfor
```

最坏情况和平均情况分析

- 对很多问题，算法性能不仅取决于输入的规模，也取决于输入的形式
- 最坏情况，平均情况，最优情况

输入规模

- 排序和搜索：数组或表中的元素的个数
- 图：图中的顶点数及边数
- 计算几何：点数，边数，线段数，多边形的数目等
- 矩阵运算：矩阵的维数
- 数论和密码学：比特数

第一个简单的例子

输入：两个整数n1和n2
输出：两个整数之和

1. sum=n1+n2
2. 输出sum

第一个简单的例子

输入：两个整数n1和n2
输出：两个整数之和

1. sum=n1+n2
2. 输出sum

数值范围？
直接利用编程语言提供的运算符？

第二个简单的例子

输入：大于等于2的正整数n
输出：n是否为素数

```
for i=2 to √n
    if(n%i==0) return false;
end for
return true
```

编程时注意的问题

将由小写字母构成的字符串str转换为大写字母

```
for(i=0;i<strlen(str);i++)
{
    str[i]=str[i]-'a'+'A';
}

for(i=0;i<str.length();i++)  //或str.size()
{
    str[i]=str[i]-'a'+'A';
}
```

对数

$$\log_b x\,y = \log_b x + \log_b y$$

$$\log_b(c^y) = y\log_b c$$

$$\log_a b = \frac{1}{\log_b a} \qquad \log_b x = \frac{\log_a x}{\log_a b}$$

$$a^{\log_a x} = x \qquad x^{\log_b y} = y^{\log_b x}$$

$$\sum_{i=1}^{n}\lfloor\log i\rfloor = (n+1)\lfloor\log n\rfloor - 2^{\lfloor\log n\rfloor+1} + 2 = \Theta(n\log n)$$

$$e = \lim_{n\to\infty}(1+\frac{1}{n})^n = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots$$

顶函数和底函数

- 若x为实数，则x的底函数$\lfloor x \rfloor$ 为不大于x的最大整数，x的顶函数$\lceil x \rceil$ 为不小于x的最小整数
- $\lceil x/2 \rceil+\lfloor x/2 \rfloor=x$　　x为整数
- $\lfloor -x \rfloor=-\lceil x \rceil, \lceil -x \rceil=-\lfloor x \rfloor$
- $x-1<\lfloor x \rfloor<=x<=\lceil x \rceil<x+1$
- 定理：f(x)为单调递增函数，若f(x)是整数时x必为整数，则$\lfloor f(\lfloor x \rfloor) \rfloor=\lfloor f(x) \rfloor$ and $\lceil f(\lceil x \rceil) \rceil=\lceil f(x) \rceil$

阶乘

- 0!=1
- n!=n(n-1)! n>=1

- Stirling公式

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n (1 + O(1/n)) \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

$$\log(n!) = \Theta(n \log n)$$

二项式系数

$$C_n^k = \frac{n!}{k!\,(n-k)!}$$

$$\binom{n}{k}$$

$$\binom{n}{k} = \binom{n}{n-k} \qquad \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

$$(1+x)^n = \sum_{j=0}^{n} \binom{n}{j} x^j$$

Pascal三角形

鸽巢原理

- 定理：如果要把n个球放入m个盒子中，则
  1) 存在一个盒子，至少装了$\lceil n/m \rceil$个球
  2) 存在一个盒子，至多装了$\lfloor n/m \rfloor$个球

和式

$$\sum_{j=1}^{n} a_{f(j)} \quad \text{or} \quad \sum_{1 \le j \le n} a_{f(j)}$$

$$\sum_{j=1}^{n} j = \frac{n(n+1)}{2} = \Theta(n^2) \qquad \sum_{j=1}^{n} j^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

$$\sum_{j=0}^{n} c^j = \frac{c^{n+1}-1}{c-1} = \Theta(c^n) \quad c \neq 1$$

$$\sum_{j=1}^{n} \frac{1}{j} = \ln n + \gamma + O(1/n) \qquad \text{Euler常数}\gamma = 0.577\ldots$$

$$\sum_{j=0}^{n} 2^j = 2^{n+1} - 1 = \Theta(2^n) \qquad \sum_{j=0}^{n} \frac{1}{2^j} = 2 - \frac{1}{2^n} < 2 = \Theta(1)$$

$$\sum_{j=0}^{\infty} c^j = \frac{1}{1-c} = \Theta(1) \quad |c| < 1$$

$$\sum_{j=1}^{n} jc^j = \frac{nc^{n+2} - nc^{n+1} - c^{n+1} + c}{(c-1)^2} = \Theta(nc^n) \quad c \neq 1$$

$$\sum_{j=1}^{n} \frac{j}{2^j} = 2 - \frac{n+2}{2^n} = \Theta(1) \qquad \sum_{j=0}^{\infty} jc^j = \frac{c}{(1-c)^2} = \Theta(1) \quad |c| < 1$$

求和的积分近似

- f(x)是单调递减的连续函数，则

$$\int_{m}^{n+1} f(x)dx \leq \sum_{j=m}^{n} f(j) \leq \int_{m-1}^{n} f(x)dx$$

- f(x)是单调递增的连续函数，则

$$\int_{m-1}^{n} f(x)dx \leq \sum_{j=m}^{n} f(j) \leq \int_{m}^{n+1} f(x)dx$$

递推关系

- 递推关系（Recurrence relation，也称递归关系）是一种函数定义方式，其在函数的定义体中包含了函数本身。
- 最著名的递推关系

  $F(n)=F(n-1)+F(n-2)$，$F(1)=1$，$F(2)=1$
- 如果有一个F(n)的显式(或称闭形式的)表达式，就可以快速计算出F(n)并与其它函数相比较。由递推关系求出显式表达式这个过程称为求解递推关系。

巧妙地猜测

- 猜测一个问题的解是一个科学的方法吗?

  $T(2n) \le 2T(n)+2n-1$, $T(2)=1$

  $T(n)=O(n\log n)$

- 一个常见的错误是试图证明相反的一面
- 另一个错误是当进行猜测时使用符号O

- 注：诸如O(n)这种形式可能蕴含很大的系数和常数项，从而c*n+O(n)<=c*n这个不等式不一定成立
- 要证明T(n)=O(n)，需要通过证明T(n)<=cn来完成

线性齐次递推式

- 线性齐次递推式：$f(n)=a_1f(n-1)+a_2f(n-2)+\ldots+a_kf(n-k)$
- $f(n)$被称为是k阶的
- 求解方法：特征方程

分治关系

- 在分治法中，问题通常被分割成几个子问题，其中每一个子问题也将被递归求解，得到结果后，再使用一个操作来组合子问题的解以得到原始问题的解。假设有a个子问题，每个问题的规模是原始问题的1/b，并且用于组合各子问题解的算法运行时间是cn^k，其中a，b，c和k是常数，即
- $T(n)=aT(n/b)+cn^k$
- 定理3.4：常数a和b为整数，a≥1，b≥2，且c和k是正常数，则递推关系$T(n)=aT(n/b)+cn^k$的解是

$$T(n)=\begin{cases} O(n^{\log_b a}) & a>b^k \\ O(n^k \log n) & a=b^k \\ O(n^k) & a<b^k \end{cases}$$

Master Theorem

- Let $a>=1$, $c>1$ be constants, $f(n)$ be a function, $T(n)$ be defined on the negative integers by the recurrence $T(n)=aT(n/c)+f(n)$ where we intepret n/c to mean either $\lceil n/c \rceil$ or $\lfloor n/c \rfloor$. Then
  1) If $f(n)=O(n^{(\log_c a-\varepsilon)})$ for some $\varepsilon>0$, then $T(n)=\Theta(n^{\log_c a})$
  2) If $f(n) = \Theta((n^{\log_c a}) * (\log n)^k)$ for some $k>=0$, then $T(n) = \Theta((n^{\log_c a}) * (\log n)^k)$
  3) If $f(n)=\Omega(n^{(\log_c a+\varepsilon)})$ for some $\varepsilon>0$ and if $a*f(n/c)\leq kf(n)$ for some constant $k<1$ and all sufficiently large n, then $T(n)=\Theta(f(n))$
- 注：核心是$f(n)$与$n^{\log_c a}$的关系
- 注：以上三种情况不完整覆盖所有情况

涉及全部历史的递推关系

- 一个涉及全部历史的递推关系，是依赖于先前所有函数值的函数，而并非仅与一个或者几个有关。
- 历史消除法